A PROJECT REPORT

AUTOMATED REGRESSION TESTING OF THE SAUCEDEMO WEB APPLICATION USING PLAYWRIGHT AND JAVASCRIPT

Submitted by

MANREET GILL

(24BCS10479)

in partial fulfilment for the award of the degree of
BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

[NAME OF INSTITUTION / UNIVERSITY]

Month Year

BONAFIDE CERTIFICATE

Certified that this project report “Automated Regression Testing of the SauceDemo Web Application using Playwright and JavaScript” is the bonafide work of “MANREET GILL (24BCS10479)” who carried out the project work under my supervision.

SIGNATURE	**SIGNATURE**
HEAD OF THE DEPARTMENT	**SUPERVISOR**
Name, Academic designation	Name, Academic designation
Department	Department
Full address of the institution	

Submitted for the project viva-voce examination held on _____

INTERNAL EXAMINER	**EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project supervisor, Supervisor name and designation, for the guidance, encouragement and valuable suggestions given throughout this project. The regular reviews helped me to simplify the project and to concentrate on tests that I could understand, run and explain.

I am thankful to the Head of the Department, HOD name, and to the faculty of the Department of Department name, Name of institution, for providing the facilities and the academic environment needed to complete this work.

I also thank the open-source community behind Playwright, Node.js and Git, whose free tools and documentation made this project possible, and the creators of the SauceDemo application, which offers a safe public website for practising test automation.

Optional: declare any tools or AI assistance used while preparing the project, as required by your institution. Example: “AI-based assistants were used for guidance on structuring the code and the report; all code was run, checked and understood by me.”

Finally, I thank my family and friends for their constant support and motivation.

Manreet Gill

TABLE OF CONTENTS

ACKNOWLEDGEMENT.....**iii**
TABLE OF CONTENTS.....**iv**
LIST OF FIGURES.....**vi**
LIST OF TABLES.....**vii**
ABSTRACT.....**viii**
GRAPHICAL ABSTRACT.....**ix**
ABBREVIATIONS.....**x**
Chapter 1: Introduction.....**1**
1.1 Identification of Client and Need.....1
1.2 Relevant Contemporary Issues.....1
1.3 Problem Identification.....1
1.4 Task Identification.....2
1.5 Timeline.....2
1.6 Organization of the Report.....3
Chapter 2: Literature Survey.....**4**
2.1 Timeline of the Reported Problem.....4
2.2 Analysis of the Literature Consulted.....4
2.3 Proposed Solutions by Different Practitioners.....5
2.4 Summary Linking the Literature Review with the Project.....5
2.5 Problem Definition.....6
2.6 Goals and Objectives.....6
Chapter 3: Design Flow / Process.....**7**
3.1 Concept Generation.....7
3.2 Evaluation and Selection of Specifications and Features.....7
3.3 Design Constraints.....7
3.4 Analysis and Feature Finalization Subject to Constraints.....8
3.5 Design Flow: Alternative Designs.....8
3.6 Best Design Selection.....9
3.7 Implementation Plan.....10
Chapter 4: Results Analysis and Validation.....**12**
4.1 Implementation of the Design Using Modern Engineering Tools.....12
4.2 Execution and Results.....14
4.3 HTML Report.....14
4.4 Analysis and Interpretation of Results.....15
4.5 Issues Faced and Their Resolution.....17
4.6 Project Management and Communication.....17

Chapter 5: Conclusion and Future Work.....19
5.1 Conclusion.....19
5.2 Deviation from Expected Results.....19
5.3 Limitations.....19
5.4 Future Work.....20
REFERENCES.....21
Appendix-1: User Manual.....22
A1.1 Requirements.....22
A1.2 Installation.....22
A1.3 Running the tests.....22
A1.4 Viewing the report.....23
A1.5 Reading the results.....24
A1.6 Common problems and solutions.....24
Appendix-2: Source Code Listings.....25
Appendix-3: Achievements.....29

LIST OF FIGURES

Figure 1: Block diagram of the test automation framework.....10
Figure 2: Flowchart of one test run.....11
Figure 3: Terminal output of npm test: 17 passed (18.6 s).....14
Figure 4: Playwright HTML report: summary and first tests.....15
Figure 5: Playwright HTML report: remaining tests.....15
Figure 6: Duration of each test in the final run (grey: first test of a worker).....16
Figure 7: Error shown when the Playwright browser was not installed.....17
Figure 8: Running npm test (user manual, step 5).....23
Figure 9: HTML report opened at localhost:9323 (user manual, step 8).....23

LIST OF TABLES

Table 1: Project timeline.....2
Table 2: Evolution of software testing and web test automation.....4
Table 3: Literature consulted, by type.....4
Table 4: Comparison of web test automation tools.....5
Table 5: Functional and non-functional requirements.....7
Table 6: Design constraints considered.....7
Table 7: Weighted decision matrix for the tool (scores 1 to 5).....8
Table 8: Weighted comparison of the alternative designs (scores 1 to 5).....9
Table 9: Software environment.....12
Table 10: Test cases, expected results and execution results (final run).....13
Table 11: Summary of the final execution.....14
Table 12: Results by feature.....15
Table 13: Issues faced and how they were resolved.....17
Table 14: Commit history of the repository.....18
Table 15: Troubleshooting.....24

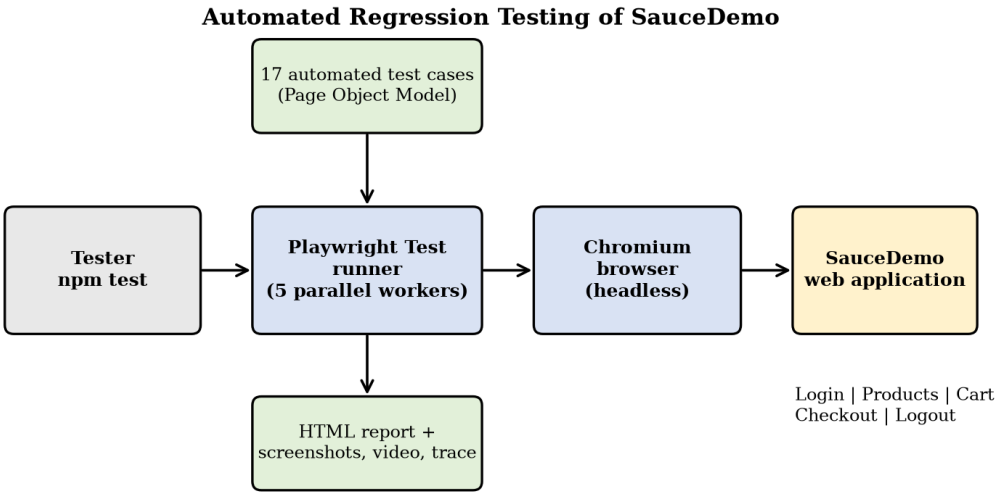
ABSTRACT

Manual regression testing of web applications is slow, repetitive and error-prone, and it becomes harder as an application changes more often. This project builds an automated regression test suite for SauceDemo, a public demonstration e-commerce website, using the Playwright Test framework and JavaScript. The suite contains 17 test cases that cover the main user journeys: login (valid, invalid, locked-out and empty credentials and access protection), product listing and sorting, cart operations, checkout with form validation, and logout.

The suite follows the Page Object Model, so locators and actions live in one class per page, while the tests only describe what is checked. Test data is kept in a single file and the specs log in first, using a simple beforeEach hook in most of them. Elements are located with stable data-test attributes and verified with web-first assertions that wait automatically, so no fixed delays are used. Tests run in parallel in headless Chromium and produce a terminal report and an HTML report with screenshots, video and traces for failures.

In the final execution all 17 tests passed with no failures in 18.6 seconds. The project also includes a test plan with test cases, a project report and a version-controlled repository on GitHub. The results show that a small, well-structured suite can verify the core functions of a web application in under twenty seconds after every change.

GRAPHICAL ABSTRACT



Graphical Abstract: Automated regression testing of SauceDemo with Playwright

ABBREVIATIONS

API	Application Programming Interface
CI/CD	Continuous Integration / Continuous Delivery
CLI	Command Line Interface
CSS	Cascading Style Sheets
DOM	Document Object Model
E2E	End-to-End (testing)
HTML	HyperText Markup Language
HTTPS	HyperText Transfer Protocol Secure
IDE	Integrated Development Environment
JS	JavaScript
JSON	JavaScript Object Notation
npm	Node Package Manager
POM	Page Object Model
QA	Quality Assurance
SUT	System Under Test
TC	Test Case
UI	User Interface
URL	Uniform Resource Locator
VCS	Version Control System
W3C	World Wide Web Consortium

CHAPTER 1
INTRODUCTION

1.1 Identification of Client and Need

The client for this project is a development or quality assurance team that maintains a customer-facing web application, represented here by SauceDemo, a public demonstration online shop. The shop offers the functions found in every e-commerce site: a login page, a product list with sorting, a shopping cart, a checkout form and a logout option. A fault in any of these functions directly affects the customers and the business of the client.

Each time the application changes, the team must confirm that the existing functions still work. Doing this by hand means repeating the same clicks and checks again and again. This is slow, tiring and error-prone, because a person may skip a step or overlook a small difference. The need is therefore a fast, repeatable way to run the same checks after every change, and to get a clear report of what passed and what failed.

1.2 Relevant Contemporary Issues

- Faster release cycles: modern teams following agile and DevOps practices release small changes often, so there is less time for manual testing before each release (Humble and Farley, 2010).
- Growing complexity of web interfaces: pages change quickly and are rendered dynamically in the browser, which makes tests that rely on fixed delays unreliable. Tools with automatic waiting reduce this problem.
- Cost of late defects: a defect found after release costs far more to correct than one found during development, so early and frequent testing is preferred (Myers, Sandler and Badgett, 2011).
- Maintainability of test code: large test suites become a burden if every test repeats the same locators. Design patterns such as the Page Object Model were introduced to keep test code maintainable (Fowler, 2013).
- Skill and tool availability: free, open-source tools now make professional-quality test automation available to students and small teams.

1.3 Problem Identification

Manual regression testing of the login, product, cart, checkout and logout functions of a web shop is repetitive and time-consuming, gives results that depend on the person doing the test, leaves no easily reviewable record, and cannot be run quickly after every

change. A student or small team also needs a suite that is simple enough to understand and maintain, not only one that works.

1.4 Task Identification

To solve the problem, the following tasks were identified:

- Study the SauceDemo application and decide which user journeys form the regression suite.
- Select a suitable test automation tool and set up the project with its dependencies.
- Design the code structure using the Page Object Model, a central test-data file and a configuration file.
- Write automated test cases with proper assertions for login, products and sorting, cart, checkout and logout.
- Execute the suite, generate reports and analyse the results.
- Prepare a test plan, test cases and this project report, and publish the work in a version-controlled repository.

1.5 Timeline

The project was carried out in iterations. The dates in Table 1 are taken from the execution records and the commit history of the repository. The first version contained 23 test cases; after a review it was simplified to 17 test cases that are easier to explain and demonstrate.

Table 1: Project timeline

Phase	Activity	Date
1	Study of the application, tool selection, project setup, page objects and first 23 tests	29 Sep 2026
2	First full execution: 23 tests passed in 15.0 seconds; code published to GitHub	29 Sep 2026
3	Review and simplification: fixture removed, 17 tests kept (commit f4a58c3)	26 Oct 2026
4	Test plan and test cases written; browser installation issue fixed	26 Oct 2026
5	Final execution: 17 tests passed in 18.6 seconds; results and report added	26 Oct 2026
6	Author name and package-lock.json added; work merged into the main branch	26 Oct 2026

1.6 Organization of the Report

The report is organised into five chapters. Chapter 1 introduces the client, the need, the problem and the tasks. Chapter 2 reviews the evolution of software testing and test automation tools and states the problem definition, goals and objectives. Chapter 3 describes the design flow: concept generation, requirements, constraints, alternative designs and the selection of the best design, with the implementation plan. Chapter 4 presents the implementation, the test results and their analysis and validation. Chapter 5 gives the conclusion, the deviation from expected results and the future work. The references and the appendices, which include a step-by-step user manual, the source code listings and the achievements, follow the last chapter.

CHAPTER 2
LITERATURE SURVEY

2.1 Timeline of the Reported Problem

Software testing as a discipline predates web applications. The classic text “The Art of Software Testing” first appeared in 1979 and set out the idea that testing is the activity of finding errors (Myers, Sandler and Badgett, 2011). As software began to change continuously, the repeated re-testing of existing functions, known as regression testing, became a major cost, and teams turned to automation. Table 2 summarises the main milestones in the evolution of web test automation.

Table 2: Evolution of software testing and web test automation

Period	Milestone
1979	First edition of “The Art of Software Testing” establishes testing principles (Myers et al., 2011 is the third edition).
2004	Selenium is created at ThoughtWorks, allowing tests to drive a real browser (Selenium Project, n.d.).
2010	“Continuous Delivery” links automated tests to frequent, reliable releases (Humble and Farley, 2010).
2013	The Page Object pattern is described as a way to keep UI test code maintainable (Fowler, 2013).
2017 (approx.)	Browser-control tools such as Puppeteer and test runners such as Cypress become widely used for JavaScript projects.
2018	W3C publishes the WebDriver specification as a Recommendation, standardising browser automation (W3C, 2018).
2020 onwards	Playwright is released by Microsoft with automatic waiting, one API for Chromium, Firefox and WebKit, and a built-in test runner (Microsoft, 2020).

2.2 Analysis of the Literature Consulted

A formal bibliometric analysis of research publications was not carried out, because the topic is driven mainly by tool documentation and industry practice rather than by journal papers. Instead, the sources consulted for this project were analysed by type. Table 3 shows that the work rests mostly on official tool documentation, together with standard books, standards and one well-known design-pattern article.

Table 3: Literature consulted, by type

Type of source	Number	Examples
Official tool documentation	4	Playwright, Selenium, Cypress, Node.js
Books	2	Myers et al.; Humble and Farley
Standards and syllabi	2	W3C WebDriver; ISTQB syllabus

Type of source	Number	Examples
Online article	1	Fowler, PageObject
Application under test	1	SauceDemo
Total	10	

2.3 Proposed Solutions by Different Practitioners

Several tools are used to automate web testing. Selenium WebDriver is the long-established standard. It supports many programming languages and all major browsers through drivers, but it is only a browser-control library: a test runner, reporting and explicit waits have to be added by the user. Puppeteer is a Node.js library from the Chrome team that controls Chromium; it is a library, not a complete test framework. Cypress is a JavaScript test framework that runs tests inside the browser, with automatic retries and an interactive runner. Playwright provides a test runner, assertions, automatic waiting, parallel execution, tracing and reports in one package, and controls Chromium, Firefox and WebKit through one API (Microsoft, 2020; Selenium Project, n.d.; Cypress.io, n.d.).

Table 4: Comparison of web test automation tools

Feature	Selenium WebDriver	Puppeteer	Cypress	Playwright
Languages	Java, Python, C#, JavaScript, others	JavaScript / TypeScript	JavaScript / TypeScript	JavaScript / TypeScript, Python, Java, .NET
Browsers	All major, via drivers	Chromium (mainly)	Chromium-family, Firefox	Chromium, Firefox, WebKit
Waiting	Explicit / implicit waits written by the user	Manual waits	Automatic retries	Automatic waiting and retrying assertions
Test runner	External (for example TestNG, JUnit)	External	Built in	Built in
Reports	External	External	Built in (basic)	Built-in HTML report, trace viewer
Licence / cost	Open source	Open source	Open-source core	Open source

The table is a simplified summary of the public documentation of each tool and is used only to guide the selection in Chapter 3.

2.4 Summary Linking the Literature Review with the Project

The literature shows three ideas that this project applies. First, regression testing is best automated because it is repetitive (Myers et al., 2011; Humble and Farley, 2010). Second, UI test code stays maintainable when page knowledge is kept in page objects instead of being repeated in every test (Fowler, 2013). Third, modern tools with

automatic waiting and built-in reporting reduce the amount of supporting code a beginner has to write, which supports the aim of a simple, explainable project. The project therefore combines Playwright with the Page Object Model on a public demonstration shop.

2.5 Problem Definition

To design and implement a simple, maintainable, automated regression test suite for the main user journeys of the SauceDemo web application (login, products and sorting, cart, checkout and logout) that runs with a single command, verifies the results with proper assertions, and produces reports that show clearly which tests passed or failed.

2.6 Goals and Objectives

The goal of the project is to replace repeated manual checking of the core functions of SauceDemo with automated tests. The objectives are:

- To set up a Playwright and JavaScript project with a clean, beginner-friendly structure.
- To automate 17 test cases covering login, products and sorting, cart, checkout and logout.
- To use the Page Object Model, a central test-data file and stable data-test locators.
- To verify results with auto-waiting assertions and without fixed delays.
- To generate a terminal report and an HTML report, with screenshots, video and traces for failures.
- To document the work with a test plan, test cases and this report, and to keep it under version control.

CHAPTER 3
DESIGN FLOW / PROCESS

3.1 Concept Generation

Two decisions had to be made: which tool to use and how to structure the test code. For the tool, four candidates from the literature survey were considered: Selenium WebDriver, Puppeteer, Cypress and Playwright. For the structure of the code, three concepts were generated: independent flat scripts, Page Objects with a custom login fixture, and Page Objects with a simple beforeEach hook.

3.2 Evaluation and Selection of Specifications and Features

The requirements were collected from the problem definition and are listed in Table 5. Functional requirements describe what the suite must check; non-functional requirements describe the quality of the suite itself.

Table 5: Functional and non-functional requirements

ID	Type	Requirement
FR1	Functional	Verify login with valid, wrong, locked-out and empty credentials, and block direct access to the inventory page without login.
FR2	Functional	Verify that 6 products are listed and that sorting by name (A-Z) and by price (low-high) puts the expected product first.
FR3	Functional	Verify adding and removing items, the cart badge count and the cart page contents.
FR4	Functional	Verify a complete purchase (overview page and confirmation message) and the required-field messages for first name and postal code.
FR5	Functional	Verify logout and return to the login page.
NFR1	Non-functional	Simple code that a beginner can read, run and explain.
NFR2	Non-functional	Tests independent of each other and free of fixed delays.
NFR3	Non-functional	Run with one command and finish in well under one minute.
NFR4	Non-functional	Produce a readable report, with evidence (screenshot, video, trace) for failures.

3.3 Design Constraints

Table 6 lists the constraints that were considered, as required for a design project. Several of them apply only lightly to a software testing project and are marked accordingly.

Table 6: Design constraints considered

Constraint	How it was considered
Regulations	Tests are run only against SauceDemo, a public site meant for practice, and respect its intended use. No other website is tested.

Constraint	How it was considered
Economic	Only free, open-source tools are used, so the cost of the tools is zero. Execution needs only a normal laptop.
Environmental	Tests run headless on one machine in about twenty seconds, so energy and resource use are minimal.
Health	No direct health aspect. Automation reduces repetitive manual work.
Manufacturability	Not applicable, because the product is software. The suite is reproducible from the repository with three commands.
Safety	The tests do not touch real customer data, payments or production systems. Only demonstration accounts are used.
Professional	Version control, readable naming (TC01 to TC17), comments and separation of data, pages and tests follow common professional practice.
Ethical	Results are reported exactly as observed, tools used are declared, and no real personal data is used.
Social	Reliable web shops give a better and safer experience to their customers; automated checks support this.
Political	Not applicable.

3.4 Analysis and Feature Finalization Subject to Constraints

Considering the constraints, the features were finalised as follows. The suite uses only the standard demonstration users and the public site. It uses open-source tools only. Tests that need complicated JavaScript logic or special setup code were avoided so that every test can be explained and typed from memory. The 23 test cases of the first version were reduced to 17 by removing tests that repeated the same pattern (for example, the last-name check repeats the first-name check) and tests that needed sorting mathematics. The features that remained are the ones that cover each function once and clearly.

3.5 Design Flow: Alternative Designs

3.5.1 Alternative tools

The four tools in Table 4 were scored on the criteria in Table 7. The weights express the priorities of this project: reliability and ease of learning matter most. The scores from 1 (poor) to 5 (very good) are the author’s assessment based on the public documentation and on the requirements NFR1 to NFR4; they are a decision aid and not a measurement.

Table 7: Weighted decision matrix for the tool (scores 1 to 5)

Criterion	Weight %	Selenium	Puppeteer	Cypress	Playwright
Ease of learning	20	3	3	4	4
Setup effort	15	2	4	4	5
Built-in reporting	15	2	2	4	5
Reliability (auto-wait)	20	3	3	4	5
Browser coverage	10	5	2	3	5

Criterion	Weight %	Selenium	Puppeteer	Cypress	Playwright
Cost	10	5	5	5	5
Documentation / community	10	5	4	4	4
Weighted total (out of 5)	100	3.30	3.20	4.00	4.70

3.5.2 Alternative designs of the test suite

Three designs of the test code were compared.

- Design 1: flat scripts. Every test contains its own locators and login steps. It is very easy to read, but the same locators are repeated everywhere and one UI change would need many edits.
- Design 2: Page Objects with a custom fixture. Locators live in page classes and a Playwright fixture logs in before each test. It is the most reusable, but fixtures are an advanced idea that is hard to explain to a beginner. This design was implemented first, as the original 23-test version.
- Design 3: Page Objects with a beforeEach hook. Locators live in page classes and the specs log in with a plain beforeEach hook (the single logout test logs in at its start). It keeps the maintainability of Page Objects while using only simple ideas.

3.6 Best Design Selection

Table 8 compares the three designs. The weights favour ease of explaining and readability because the project must be demonstrated and defended, without ignoring maintainability.

Table 8: Weighted comparison of the alternative designs (scores 1 to 5)

Criterion	Weight %	Design 1	Design 2	Design 3
Readability for a beginner	25	4	3	5
Maintainability	25	1	5	4
Reuse / less duplication	20	1	5	4
Ease of explaining and demonstrating	30	4	2	5
Weighted total (out of 5)	100	2.65	3.60	4.55

Playwright obtained the highest weighted total among the tools (4.70), mainly because it provides automatic waiting, a built-in test runner and an HTML report without extra libraries. Design 3 obtained the highest total among the designs (4.55), because it is almost as maintainable as Design 2 but far easier to explain. Playwright with the Page Object Model and a beforeEach hook was therefore selected. The scores are

judgements. The ranking among the tools holds for any choice of weights, because Playwright scores at least as high as every other tool on every criterion; Design 3 stays first as long as readability and ease of explaining carry a large share of the weight, as they do in this project.

3.7 Implementation Plan

Figure 1 shows the block diagram of the selected design. The test specifications call the page objects, which use the Playwright API to drive a headless Chromium browser against SauceDemo. Test data and settings are kept in separate files, and the reporters collect the results.

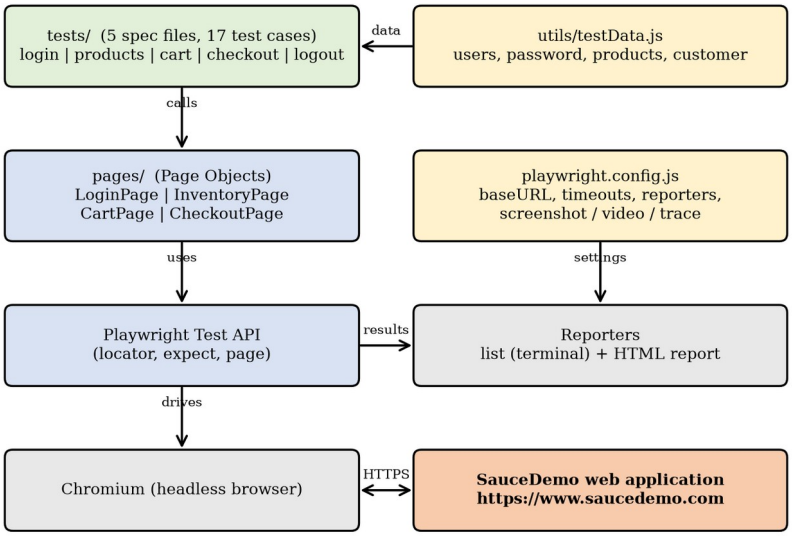


Figure 1: Block diagram of the test automation framework

Figure 2 shows the flowchart of a test run. The algorithm followed by every test is:

- Step 1: Playwright reads the configuration and finds the spec files.
- Step 2: A worker starts, launches Chromium and creates a fresh browser context for the test.
- Step 3: The beforeEach hook opens the site and logs in (and, for checkout, adds an item and opens the checkout form).
- Step 4: The test performs its actions through the page objects.
- Step 5: An expect(...) assertion compares the actual result with the expected result and retries for up to 5 seconds.
- Step 6: If the condition is met the test passes; otherwise it fails and a screenshot, video and trace are saved.
- Step 7: All results are collected into the terminal output and the HTML report.

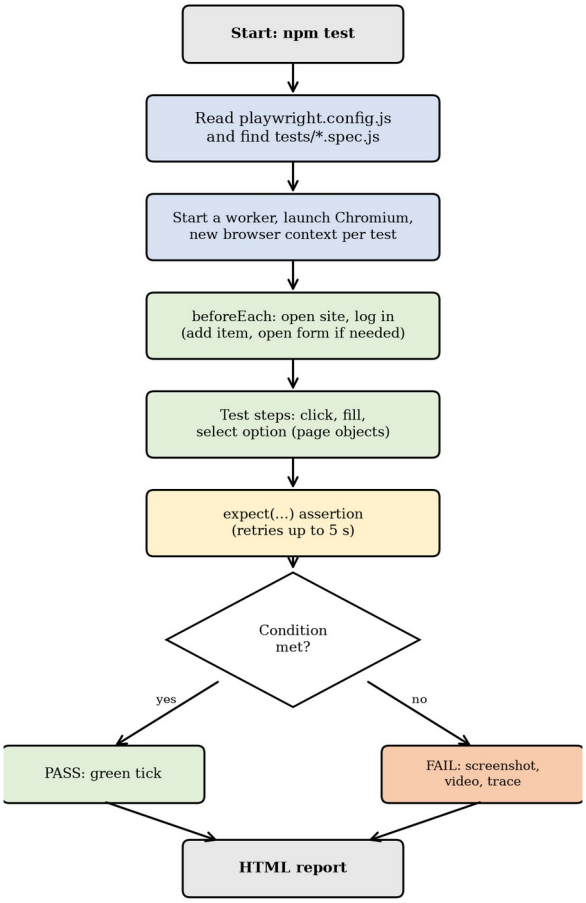


Figure 2: Flowchart of one test run

CHAPTER 4
RESULTS ANALYSIS AND VALIDATION

4.1 Implementation of the Design Using Modern Engineering Tools

4.1.1 Environment and tools

The suite was developed and executed on a Windows laptop using the tools in Table 9. Playwright downloads its own Chromium build, so no separate browser installation or driver is needed.

Table 9: Software environment

Item	Detail
Operating system	Windows
Runtime and package manager	Node.js (LTS) and npm
Test framework	@playwright/test (version 1.63 series)
Browser	Chromium installed by Playwright, headless mode
Editor	Visual Studio Code with integrated terminal
Version control	Git, with the repository hosted on GitHub
Application under test	SauceDemo, https://www.saucedemo.com

4.1.2 Project structure

The project is organised so that each folder has one purpose. The listing below shows the structure of the final repository.

```
saucedemo-regression-tests/
|-- playwright.config.js   settings: base URL, timeouts, reporters, evidence
|-- package.json           dependencies and npm scripts
|-- package-lock.json      exact dependency versions
|-- pages/                 Page Objects (LoginPage, InventoryPage, CartPage,
CheckoutPage)
|-- tests/                 login, products, cart, checkout, logout specs
|-- utils/testData.js      users, products, customer details
`-- docs/                  test plan, test cases and project report
```

4.1.3 Configuration

The configuration file sets the test folder, a 30 second test timeout, a 5 second assertion timeout and no retries, so that a failure is never hidden. It enables the list and HTML reporters and keeps a screenshot, video and trace only for failed tests. The base URL is set once so that tests use relative paths such as “/”. The complete file is given in Appendix 2.

4.1.4 Page Objects

Each page object holds the locators of one page and the actions a user can perform. As an example, the login page object has locators for the username, password, login button and error message, and the methods goto() and login(). The tests do not contain selectors; they call these methods. Locators use the data-test attributes that SauceDemo provides for testing, which are more stable than CSS classes or visible text.

4.1.5 Test design and assertions

Every test follows the same pattern: perform an action, then check one thing. The checks use web-first assertions such as toHaveURL, toHaveText, toContainText, toHaveCount and toBeVisible. These assertions retry until the condition is true or the 5 second timeout ends, so there are no fixed delays. Most spec files use a beforeEach hook: the login spec opens the login page, the products and cart specs log in, and the checkout spec also adds the backpack to the cart and opens the checkout form. The single logout test logs in at its start. Table 10 lists all 17 test cases.

Table 10: Test cases, expected results and execution results (final run)

ID	Feature	Scenario	Expected / actual result	Status	Time (s)
TC01	Login	Valid login	Inventory page opens; title “Products”	Pass	8.8
TC02	Login	Wrong password	Error: username and password do not match	Pass	2.6
TC03	Login	Locked-out user	Error: this user has been locked out	Pass	2.5
TC04	Login	Empty username	Error: “Username is required”	Pass	1.1
TC05	Login	Empty password	Error: “Password is required”	Pass	1.1
TC06	Login	Inventory page without login	Error: only accessible when logged in	Pass	1.1
TC07	Products	Product listing	Exactly 6 products shown	Pass	8.5
TC08	Products	Sort name A to Z	First product: Sauce Labs Backpack	Pass	2.4
TC09	Products	Sort price low to high	First product: Sauce Labs Onesie	Pass	2.7
TC10	Cart	Add one item	Cart badge shows 1	Pass	8.9
TC11	Cart	Add two items	Cart badge shows 2	Pass	2.6
TC12	Cart	Remove item	Cart badge disappears	Pass	2.7
TC13	Cart	View cart	Cart page lists the added item	Pass	1.2
TC14	Checkout	Complete purchase	Overview page opens; “Thank you for your order!”	Pass	8.8
TC15	Checkout	First name missing	Error: “First Name is required”	Pass	2.3
TC16	Checkout	Postal code missing	Error: “Postal Code is required”	Pass	2.5
TC17	Logout	Logout	Login button visible (login page shown)	Pass	9.1

4.2 Execution and Results

The suite was executed with the command npm test. Playwright ran the 17 tests with 5 parallel workers in headless Chromium. Figure 3 shows the terminal output of the final run: every test is marked with a green tick and the summary reads “17 passed (18.6s)”. The result is also summarised in Table 11.

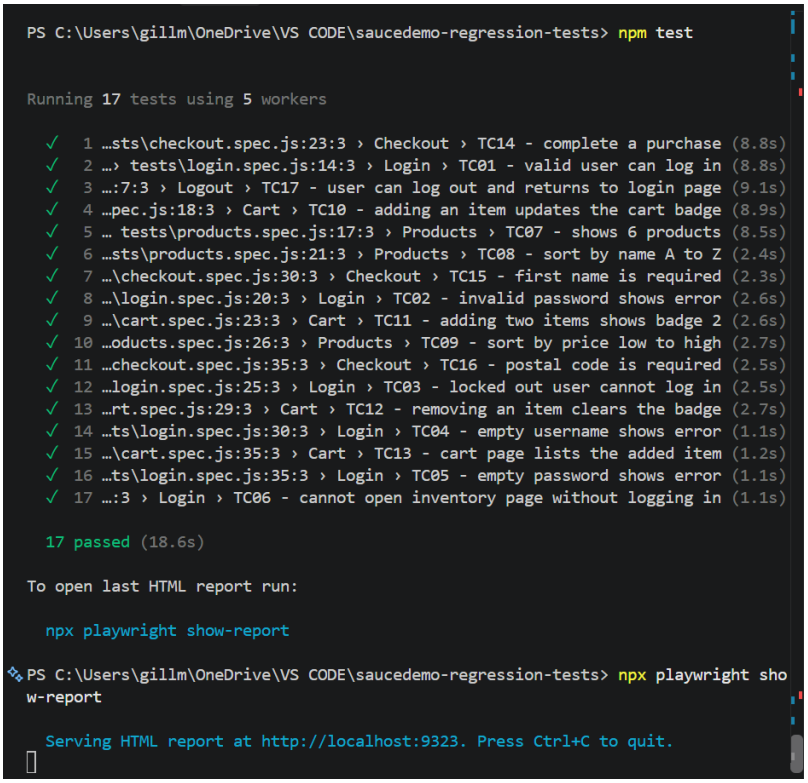


Figure 3: Terminal output of npm test: 17 passed (18.6 s)

Table 11: Summary of the final execution

Item	Result
Browser / mode	Chromium, headless
Parallel workers	5
Tests executed	17
Passed	17
Failed / flaky / skipped	0 / 0 / 0
Total execution time	18.6 seconds
Run date	26 October 2026

4.3 HTML Report

After the run, the command npm run report serves the HTML report at http://localhost:9323. Figure 4 shows the summary bar with All 17, Passed 17, Failed 0, Flaky 0 and Skipped 0, followed by the tests grouped by spec file with their durations. Figure 5 shows the remaining tests of the report.

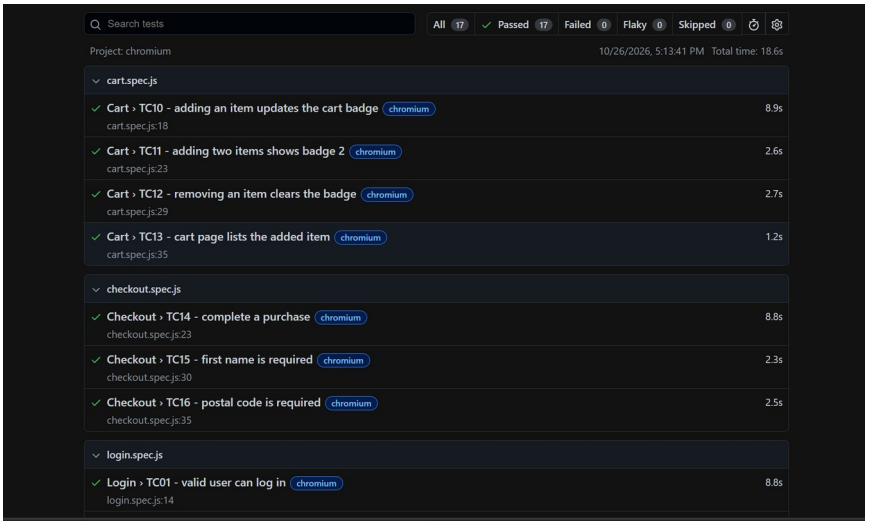


Figure 4: Playwright HTML report: summary and first tests

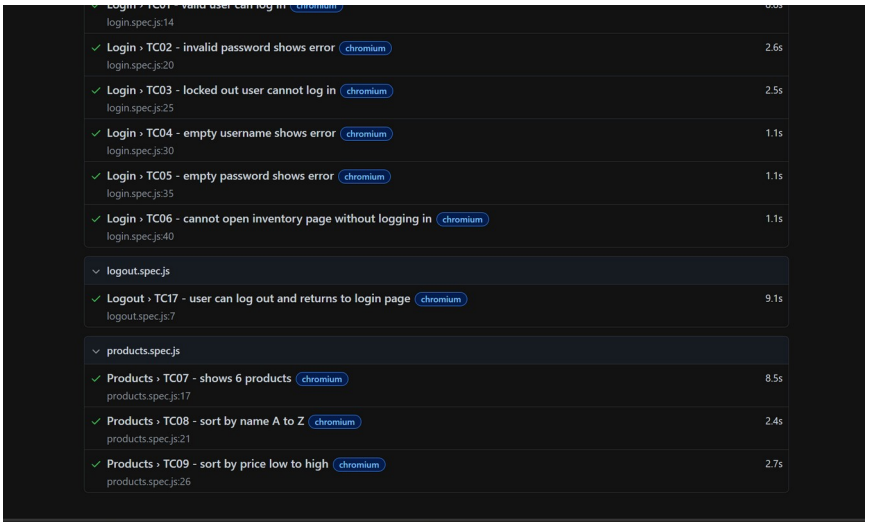


Figure 5: Playwright HTML report: remaining tests

4.4 Analysis and Interpretation of Results

4.4.1 Results by feature

Table 12 groups the results by feature. All tests of every feature passed. The sum of the times is larger than the total of 18.6 seconds because five tests ran at the same time.

Table 12: Results by feature

Feature	Tests	Passed	Failed	Sum of times (s)	Average (s)
Login	6	6	0	17.2	2.9
Products	3	3	0	13.6	4.5
Cart	4	4	0	15.4	3.8
Checkout	3	3	0	13.6	4.5
Logout	1	1	0	9.1	9.1

4.4.2 Test duration

Figure 6 shows the duration of every test. Five tests took between 8.5 and 9.1 seconds (TC01, TC07, TC10, TC14 and TC17), whereas the others took between 1.1 and 2.7 seconds. These five were the first tests started by each of the five workers. The likely reason is the start-up of the browser and the first page load in each worker, which is paid only once per worker. This is an interpretation of the pattern in the data, not a separate measurement. The quick tests TC04, TC05, TC06 and TC13 took about one second because they do very little after loading the page.

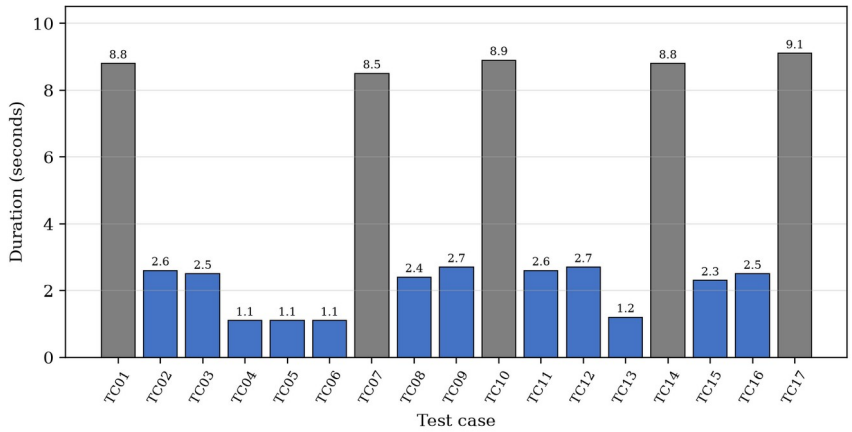


Figure 6: Duration of each test in the final run (grey: first test of a worker)

4.4.3 Data validation

- Positive tests (TC01, TC14, TC17) confirm that valid actions succeed: login reaches the Products page, a purchase ends with the confirmation message, and logout returns to the login page.
- Negative tests (TC02 to TC06, TC15, TC16) confirm that invalid input is rejected with the correct message, so a wrong password, a locked-out user, an empty field or direct URL access is not accepted.
- Exact values are checked where the application defines them: 6 products, the first product after sorting A-Z (Sauce Labs Backpack) and after sorting by price low-high (Sauce Labs Onesie), the overview page address and the confirmation message.
- Independence: each test starts in a new browser context with an empty cart, so the order of the tests does not matter and they can run in parallel.

A useful way to confirm that a test really can fail is to change one expected value, for example “Products” to “Product” in TC01, and run it again. The test should then fail and the report should show a screenshot. This check is recommended before a demonstration.

4.5 Issues Faced and Their Resolution

Table 13: Issues faced and how they were resolved

Issue	Resolution
npm reported “Could not read package.json”	The project was unzipped into a folder inside another folder, so npm ran in the wrong directory. The terminal was moved into the inner folder that contains package.json.
All tests failed with “Executable doesn’t exist” in about 1 to 6 ms each (Figure 7)	The browser build required by the installed Playwright version was not downloaded. Running npx playwright install chromium fixed it. These runs were not counted as results.
git push failed with a 400 error	The remote URL still contained a placeholder user name. It was corrected with git remote set-url.
git push was rejected (fetch first)	The new repository was not empty because a file had been uploaded. The remote was replaced with the project using a forced push, which was safe in that situation.
git pull failed with “Unlink of file failed”	A Word document in the folder was open and the folder was synchronised by OneDrive. Closing the file, pausing synchronisation and pulling again solved it.
Suite hard to explain	The fixture and the sorting mathematics were removed and the 23 tests were reduced to 17 simple tests (Design 3).

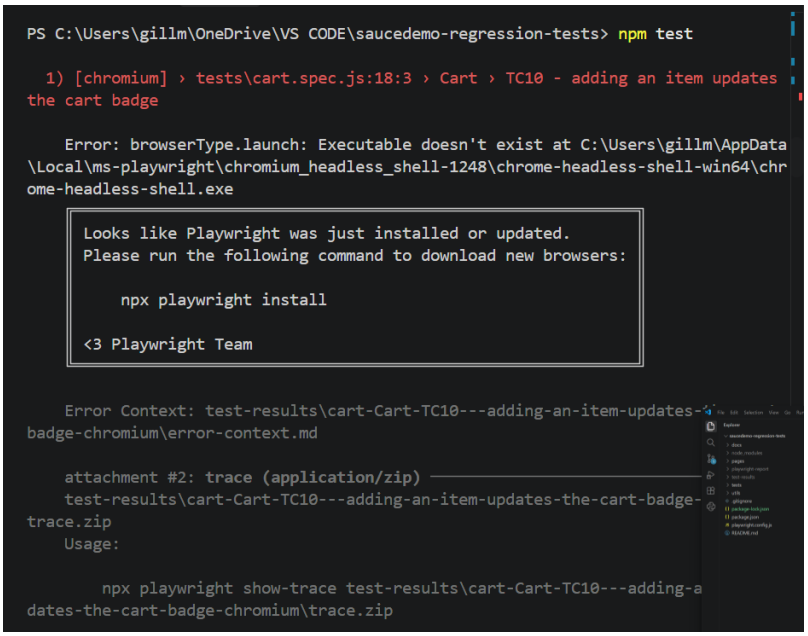


Figure 7: Error shown when the Playwright browser was not installed

4.6 Project Management and Communication

The work was managed with Git. Each change was committed with a message that describes it, and the repository was published on GitHub so that it could be reviewed. Table 14 lists the commit history of the main branch. The review feedback led to the simplification of the suite, the addition of the test plan and test cases, and the housekeeping fixes (author name and lock file).

Table 14: Commit history of the repository

Commit	Date	Message
fa51b4a	29 Sep 2026	SauceDemo regression tests (first version, 23 tests)
f4a58c3	26 Oct 2026	Simplify suite to 17 easy-to-explain tests; remove fixture and sorting maths
d4d1635	26 Oct 2026	Add test plan and test cases
cdda81f	26 Oct 2026	Fill test plan with real results (17 passed, 0 failed, 18.6 s)
deed233	26 Oct 2026	Add project report
b0e6a22	26 Oct 2026	Set author in package.json
735fc33	26 Oct 2026	Add package-lock.json so installs use the same versions

CHAPTER 5
CONCLUSION AND FUTURE WORK

5.1 Conclusion

The project designed and implemented an automated regression test suite for the SauceDemo web application using Playwright and JavaScript. The suite contains 17 test cases that cover login, products and sorting, cart, checkout and logout, and it is organised with the Page Object Model, a central test-data file and a configuration file. In the final execution all 17 tests passed, none failed, and the whole suite finished in 18.6 seconds, which shows that the core functions of the application can be re-verified in less than twenty seconds after any change.

The project met its objectives: the project structure is clean and simple, the tests use auto-waiting assertions without fixed delays, the reports provide clear evidence, and the work is documented with a test plan, test cases and this report and is stored in a version-controlled repository. An important lesson was that a simpler design that can be explained is more valuable for a student project than a more advanced one: the first version with a custom fixture was replaced by a simple beforeEach hook without any loss of coverage of the main journeys.

5.2 Deviation from Expected Results

The test results matched the expected results: no test failed in the final run. Two deviations from the original plan are worth recording. First, the first version of the suite contained 23 test cases and was reduced to 17 after the review, which lowers the coverage of repeated patterns (for example, last-name validation, sort Z-A and price high-low, and the cart reset after an order) in exchange for simplicity. Second, early runs on the new copy of the project failed because the Playwright browser had not been downloaded; these were set-up failures, not test results, and were solved with one command.

5.3 Limitations

- The suite runs only on Chromium and only for the standard and locked-out demo users.
- The sort tests check only the first product after sorting, not the whole list.
- Several tests depend on the current demonstration data (6 products and the first product after each sort), so a change in the site data would require updates.
- The tests need internet access to the public SauceDemo site.

5.4 Future Work

- Add Firefox and WebKit projects to the configuration to test cross-browser behaviour.
- Add tests for other demo users, such as problem_user and performance_glitch_user, to find user-specific defects.
- Add the removed tests back in a simple form: sort Z-A and price high-low, last-name validation and cart reset after an order.
- Run the suite automatically on every push using GitHub Actions (continuous integration).
- Compare the whole sorted list, add visual comparison and add API-level tests.

REFERENCES

1. Cypress.io (n.d.) Cypress Documentation. Available at: https://docs.cypress.io
2. Fowler, M. (2013) ‘PageObject’, martinfowler.com. Available at: https://martinfowler.com/bliki/PageObject.html
3. Humble, J. and Farley, D. (2010) Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Boston: Addison-Wesley.
4. ISTQB (2023) Certified Tester Foundation Level Syllabus. International Software Testing Qualifications Board.
5. Microsoft (2020) Playwright Documentation. Available at: https://playwright.dev
6. Myers, G.J., Sandler, C. and Badgett, T. (2011) The Art of Software Testing. 3rd edn. Hoboken, NJ: John Wiley & Sons.
7. OpenJS Foundation (n.d.) Node.js Documentation. Available at: https://nodejs.org/docs
8. Sauce Labs (n.d.) SauceDemo (Swag Labs) demonstration application. Available at: https://www.saucedemo.com
9. Selenium Project (n.d.) Selenium Documentation. Available at: https://www.selenium.dev/documentation/
10. W3C (2018) WebDriver, W3C Recommendation. Available at: https://www.w3.org/TR/webdriver/

APPENDIX-1
USER MANUAL

This manual explains how to set up and run the project on a Windows computer. All commands are typed in the terminal of Visual Studio Code (Terminal, then New Terminal), inside the project folder.

A1.1 Requirements

- Node.js (LTS version) from nodejs.org, which includes npm.
- Git, from git-scm.com, and a GitHub connection to download the code.
- Visual Studio Code (recommended) and an internet connection.

A1.2 Installation

Step 1: Open a terminal and download the project: git clone https://github.com/gillmreet01/saucedemo-regression-tests.git

Step 2: Go into the folder: cd saucedemo-regression-tests

Step 3: Install the dependencies: npm install

Step 4: Download the browser used by the tests: npx playwright install chromium

If the second command is skipped, the tests fail at once with the message “Executable doesn’t exist” (Figure 7). Running npx playwright install chromium removes the problem.

A1.3 Running the tests

Step 5: Run all 17 tests: npm test. The terminal lists every test with a green tick and ends with the line “17 passed” (Figure 8).

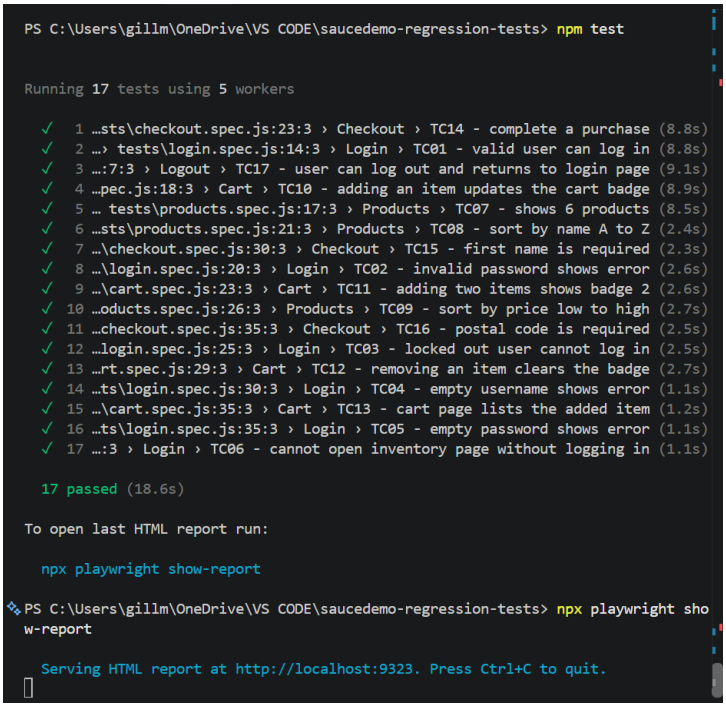


Figure 8: Running npm test (user manual, step 5)

Step 6: To watch the browser while the tests run, use npm run test:headed.

Step 7: To run only one feature, use npm run test:login, test:products, test:cart, test:checkout or test:logout.

A1.4 Viewing the report

Step 8: Open the HTML report with npm run report. The terminal prints “Serving HTML report at http://localhost:9323”. Open this address in a browser (Figure 9).

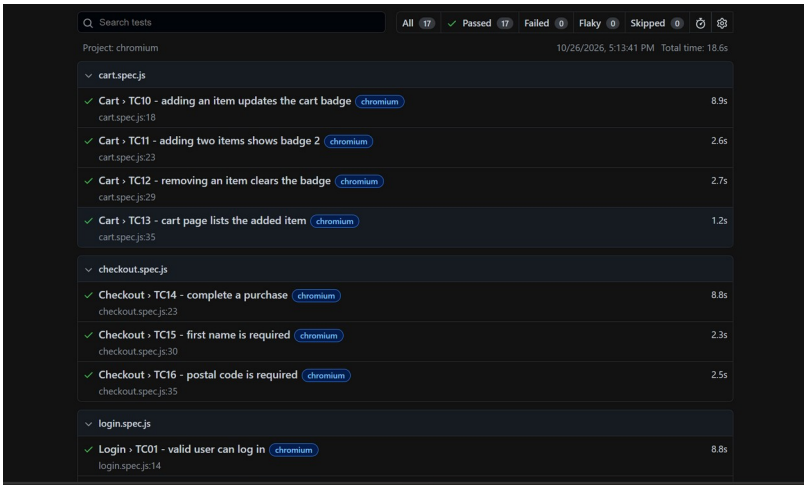


Figure 9: HTML report opened at localhost:9323 (user manual, step 8)

Step 9: Press Ctrl+C in the terminal to stop the report server.

A1.5 Reading the results

- A green tick means the test passed; a red cross means it failed.
- For a failed test, the HTML report shows the error, a screenshot, a video and a trace of the steps.
- Results from the final run are summarised in Table 11 and Table 10 of Chapter 4.

A1.6 Common problems and solutions

Table 15: Troubleshooting

Problem	Solution
“Could not read package.json”	The terminal is in the wrong folder. Move into the folder that contains package.json.
“Executable doesn’t exist”	Run npx playwright install chromium and run npm test again.
Tests fail with network errors	Check that https://www.saucedemo.com opens in a normal browser and that the internet connection works.
git pull says “Unlink of file failed”	Close any open Word or PDF files in the project and pause OneDrive synchronisation, then pull again.

APPENDIX-2
SOURCE CODE LISTINGS

The key files of the final version of the project (17 tests) are listed below: the configuration, the test data, two page objects and two spec files. The remaining page objects (CartPage, CheckoutPage) and spec files (products, cart, logout) follow the same patterns and are available, with everything else, in the GitHub repository gillmreet01/saucedemo-regression-tests.

playwright.config.js

```
// Playwright configuration - controls how all tests run
const { defineConfig } = require('@playwright/test');

module.exports = defineConfig({
  testDir: './tests',            // where the test files live
  timeout: 30 * 1000,            // max time for one test
  expect: { timeout: 5 * 1000 }, // max time for one assertion to pass
  retries: 0,                    // no automatic re-runs (keeps results honest)
  reporter: [
    ['list'],                                                  // live output
in terminal
    ['html', { outputFolder: 'playwright-report', open: 'never' }], // HTML
report
  ],
  use: {
    baseURL: 'https://www.saucedemo.com',
    headless: true,
    screenshot: 'only-on-failure', // attach screenshot when a test fails
    video: 'retain-on-failure',    // keep video only for failed tests
    trace: 'retain-on-failure',    // keep trace only for failed tests
  },
  projects: [
    { name: 'chromium', use: { browserName: 'chromium' } },
  ],
});
```

utils/testData.js

```
// All test data in one place so tests stay clean.
module.exports = {
  password: 'secret_sauce',
  users: {
    standard: 'standard_user',
    locked: 'locked_out_user',
  },
  products: {
    backpack: 'Sauce Labs Backpack',
    bikeLight: 'Sauce Labs Bike Light',
  },
  customer: {
    firstName: 'Manreet',
    lastName: 'Gill',
    postalCode: '12345',
```

```
  },
};
```

pages/LoginPage.js

```
// Page Object: everything about the login page.
class LoginPage {
  constructor(page) {
    this.page = page;
    this.usernameInput = page.locator('[data-test="username"]');
    this.passwordInput = page.locator('[data-test="password"]');
    this.loginButton = page.locator('[data-test="login-button"]');
    this.errorMessage = page.locator('[data-test="error"]');
  }

  async goto() {
    await this.page.goto('/');
  }

  async login(username, password) {
    await this.usernameInput.fill(username);
    await this.passwordInput.fill(password);
    await this.loginButton.click();
  }
}

module.exports = { LoginPage };
```

pages/InventoryPage.js

```
// Page Object: the products page (also has the menu and cart icon).
class InventoryPage {
  constructor(page) {
    this.page = page;
    this.title = page.locator('[data-test="title"]');
    this.items = page.locator('[data-test="inventory-item"]');
    this.itemNames = page.locator('[data-test="inventory-item-name"]');
    this.sortDropdown = page.locator('[data-test="product-sort-container"]');
    this.cartBadge = page.locator('[data-test="shopping-cart-badge"]');
    this.cartLink = page.locator('[data-test="shopping-cart-link"]');
    this.menuButton = page.locator('#react-burger-menu-btn');
    this.logoutLink = page.locator('[data-test="logout-sidebar-link"]');
  }

  // Turns "Sauce Labs Backpack" into "add-to-cart-sauce-labs-backpack"
  idFor(productName) {
    return productName.toLowerCase().replace(/ /g, '-');
  }

  async addToCart(productName) {
    await this.page.locator(`[data-test="add-to-cart-$
{this.idFor(productName)}"]`).click();
  }

  async removeFromCart(productName) {
    await this.page.locator(`[data-test="remove-$
{this.idFor(productName)}"]`).click();
  }
```

```
  async sortBy(value) {
    await this.sortDropdown.selectOption(value); // az, za, lohi, hilo
  }

  async openCart() {
    await this.cartLink.click();
  }

  async logout() {
    await this.menuButton.click();
    await this.logoutLink.click();
  }
}

module.exports = { InventoryPage };
```

tests/login.spec.js

```
const { test, expect } = require('@playwright/test');
const { LoginPage } = require('../pages/LoginPage');
const { InventoryPage } = require('../pages/InventoryPage');
const { users, password } = require('../utils/testData');

test.describe('Login', () => {
  let loginPage;

  test.beforeEach(async ({ page }) => {
    loginPage = new LoginPage(page);
    await loginPage.goto();
  });

  test('TC01 - valid user can log in', async ({ page }) => {
    await loginPage.login(users.standard, password);
    await expect(page).toHaveURL(/inventory\.html/);
    await expect(new InventoryPage(page).title).toHaveText('Products');
  });

  test('TC02 - invalid password shows error', async () => {
    await loginPage.login(users.standard, 'wrong_password');
    await expect(loginPage.errorMessage).toContainText('Username and password do
not match');
  });

  test('TC03 - locked out user cannot log in', async () => {
    await loginPage.login(users.locked, password);
    await expect(loginPage.errorMessage).toContainText('this user has been
locked out');
  });

  test('TC04 - empty username shows error', async () => {
    await loginPage.login('', password);
    await expect(loginPage.errorMessage).toContainText('Username is required');
  });

  test('TC05 - empty password shows error', async () => {
    await loginPage.login(users.standard, '');
    await expect(loginPage.errorMessage).toContainText('Password is required');
  });
```

```
  test('TC06 - cannot open inventory page without logging in', async ({ page })
=> {
    await page.goto('/inventory.html');
    await expect(loginPage.errorMessage).toContainText("You can only access
'/inventory.html' when you are logged in");
  });
});
```

tests/checkout.spec.js

```
const { test, expect } = require('@playwright/test');
const { LoginPage } = require('../pages/LoginPage');
const { InventoryPage } = require('../pages/InventoryPage');
const { CartPage } = require('../pages/CartPage');
const { CheckoutPage } = require('../pages/CheckoutPage');
const { users, password, products, customer } = require('../utils/testData');

test.describe('Checkout', () => {
  let checkoutPage;

  // Before each test: log in, add a backpack and open the checkout form
  test.beforeEach(async ({ page }) => {
    const loginPage = new LoginPage(page);
    await loginPage.goto();
    await loginPage.login(users.standard, password);
    const inventoryPage = new InventoryPage(page);
    await inventoryPage.addToCart(products.backpack);
    await inventoryPage.openCart();
    await new CartPage(page).checkout();
    checkoutPage = new CheckoutPage(page);
  });

  test('TC14 - complete a purchase', async ({ page }) => {
    await checkoutPage.fillInfo(customer.firstName, customer.lastName,
customer.postalCode);
    await expect(page).toHaveURL(/checkout-step-two/);
    await checkoutPage.finishButton.click();
    await expect(checkoutPage.completeHeader).toHaveText('Thank you for your
order!');
  });

  test('TC15 - first name is required', async () => {
    await checkoutPage.fillInfo('', customer.lastName, customer.postalCode);
    await expect(checkoutPage.errorMessage).toContainText('First Name is
required');
  });

  test('TC16 - postal code is required', async () => {
    await checkoutPage.fillInfo(customer.firstName, customer.lastName, '');
    await expect(checkoutPage.errorMessage).toContainText('Postal Code is
required');
  });
});
```

APPENDIX-3
ACHIEVEMENTS

- Built a complete automated regression suite of 17 test cases for SauceDemo using Playwright and JavaScript.
- Achieved 17 passed and 0 failed tests in the final run, completed in 18.6 seconds with 5 parallel workers.
- Applied the Page Object Model, central test data, stable data-test locators and auto-waiting assertions without fixed delays.
- Generated terminal and HTML reports, with screenshots, video and traces kept for failed tests.
- Compared tools and designs with weighted decision matrices and selected the best design, and simplified the first 23-test version to a design that can be fully explained.
- Prepared a test plan with test cases, this project report, a one-page viva cheat sheet and practice questions.
- Published the project on GitHub with a clean commit history, a lock file and documentation in the docs folder.
- Diagnosed and solved real set-up problems: wrong folder, missing browser download, Git remote errors and file locking.